hashlock.

# Security Audit

## Immutable Ratings V2 (Voting Platform)

June 2025 info@hashlock.com.au Hashlock Pty Ltd

# Table of Contents

## CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

# Executive Summary

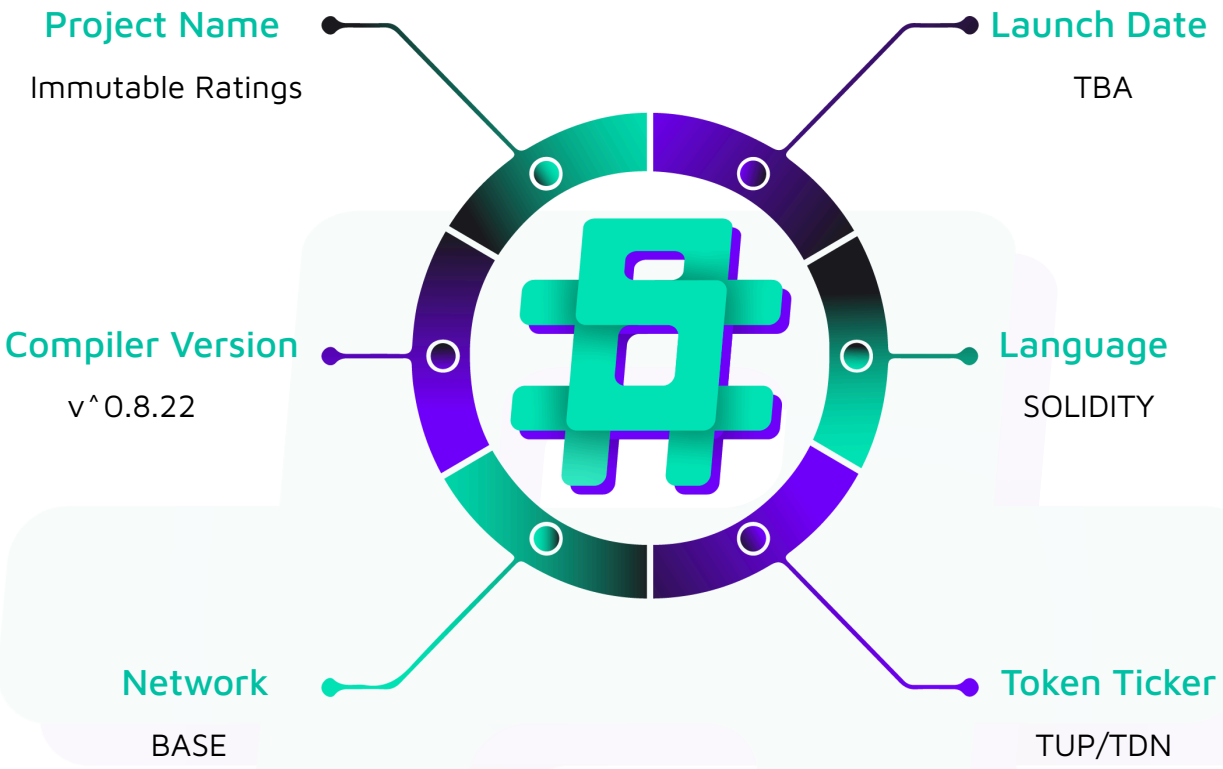
The Immutable Ratings team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

# Project Context

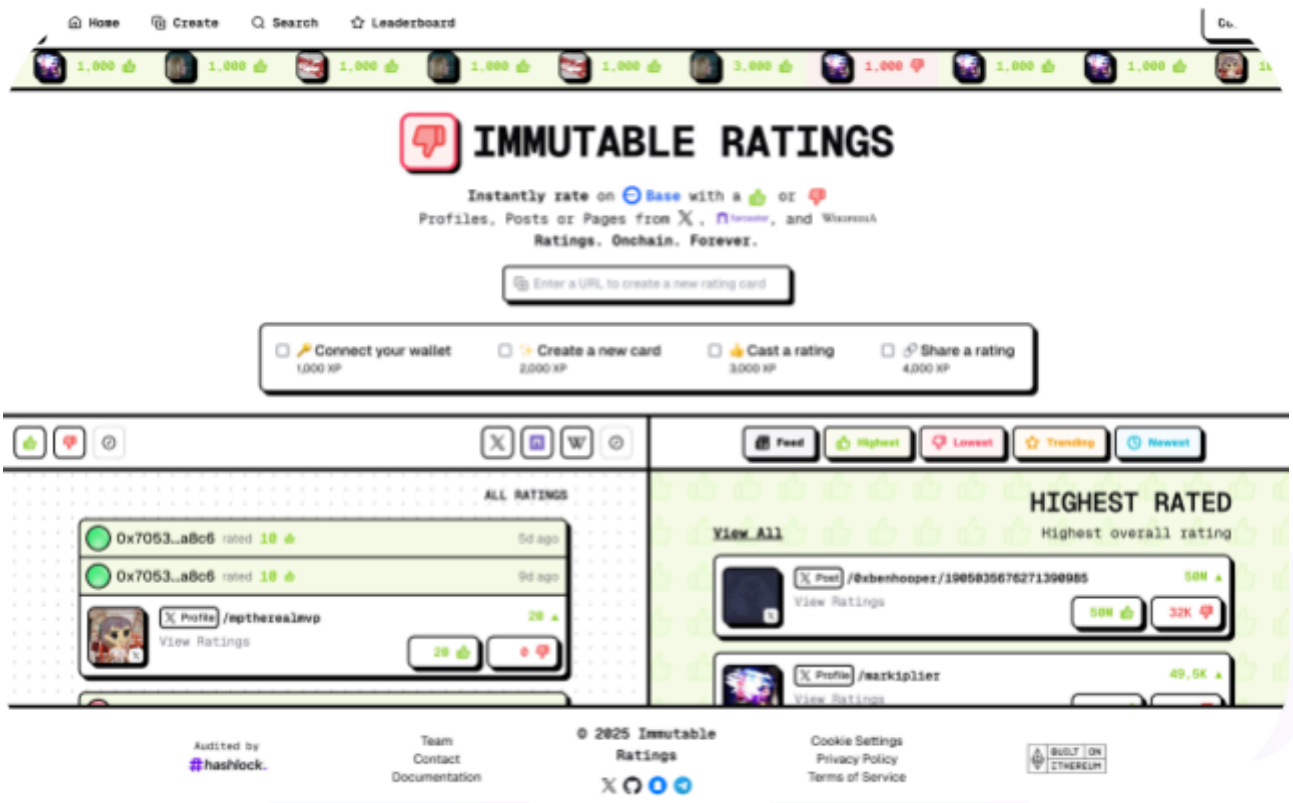
IImmutable Ratings is a decentralized on-chain reputation protocol that empowers Web3 communities to instantly cast thumbs-up or thumbs-down across major social platforms—X, Warpcast, LinkedIn, and Wikipedia—while immutably storing every vote on the Base blockchain via smart contracts. By aggregating trustless feedback into real-time feeds, trending indexes, and community-driven leaderboards (Highest Rated, Lowest Rated, Trending, Newly Added), Immutable Ratings delivers transparent, tamper-proof social signals and reputation scores. Whether you're a content creator, DAO member, or brand, you can leverage these verifiable metrics to fuel decentralized governance, on-chain credit scoring, and next-generation social analytics—ushering in a new era of Web3 credibility and community-powered discovery.

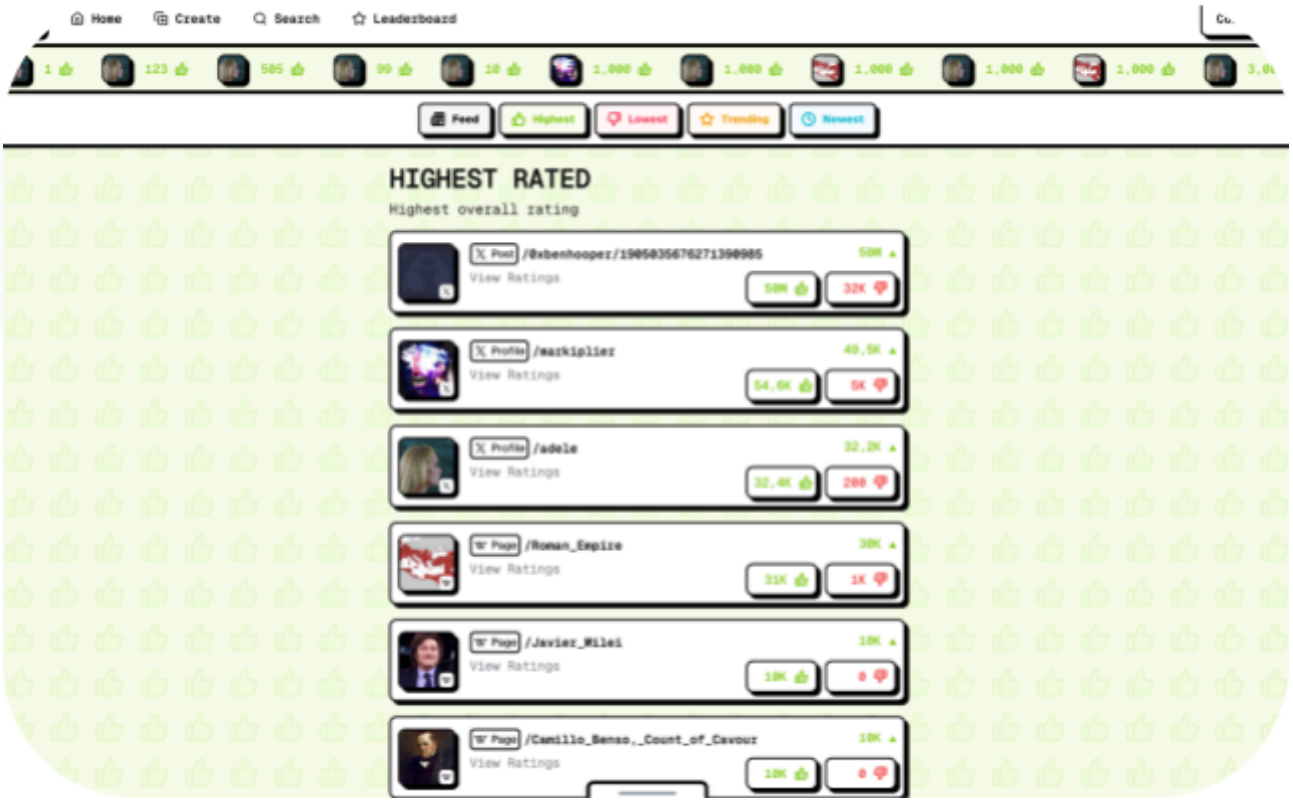
**Project Name**: Immutable Ratings
**Project Type:** Voting Platform
**Compiler Version:** ^0.8.22
**Website:** https://www.ratings.wtf
**Logo:**

**Visualised Context:**



**Project Visuals:**



# Audit Scope

We at Hashlock audited the solidity code within the Immutable Ratings project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

| **Description** | **Immutable Ratings Smart Contracts** |
|---|---|
| **Platform** | **EVM / Solidity** |
| **Audit Date** | **Jun, 2025** |
| **Contract 1** | ImmutableMapping.sol |
| **Contract 2** | ImmutableRatings.sol |
| **Contract 3** | IWETH.sol |
| **Contract 4** | IV3SwapRouter.sol |
| **Contract 5** | IRatingHook.sol |
| **Audited GitHub Commit Hash** | c560db89095b7e95ff195f43681d813d85320171 |
| **Fix Review GitHub Commit Hash** | 9923b816d95e593b6fb896b95716871798e17e2f |

# Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.


*The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.*

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

**Hashlock found:**

1 High severity vulnerability

2 Medium severity vulnerabilities

3 Low severity vulnerabilities

2 QA

**Caution:** *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

# Intended Smart Contract Functions

| Claimed Behaviour | Actual Behaviour |
|---|---|
| **ImmutableRatings.sol**<br>- Allows users to:<br>- Create a Market for a URL<br>- Upvote a Market (Create an Up Rating)<br>- Downvote a Market (Create a Down Rating)<br>- Batch Upvote/Downvote Multiple Markets<br>- View a User's Total Ratings<br>- Preview the Cost of a Rating<br>- Allows admins to:<br>- Set the Fee Receiver<br>- Pause/Unpause the Contract<br>- Distribute ETH Payments to the Receiver<br>- Allows token Swaps | **Contract achieves this functionality.** |
| **UniversalMappingProtocol.sol**<br>- Link Off Chain address to On-Chain Address | **Contract achieves this functionality.** |

# Code Quality

This audit scope involves the smart contracts of the Immutable Ratings project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

# Audit Resources

We were given the Immutable Ratings project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

# Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

# Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

| Significance | Description |
|---|---|
| **High** | High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community. |
| **Medium** | Medium-level difficulties should be solved before deployment, but won't result in loss of funds. |
| **Low** | Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future. |
| **Gas** | Gas Optimisations, issues, and inefficiencies. |
| **QA** | Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code. |

# Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

| Significance | Description |
|---|---|
| **Resolved** | The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue. |
| **Acknowledged** | The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception. |
| **Unresolved** | The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed. |

# Audit Findings

## High

### [H-01] ImmutableRatings#_processPayment - Strict Payment Validation Leads to Frequent Transaction Failures.

**Description**

The `_processPayment` function requires an exact match (`msg.value != price`) for native currency payments, which is too rigid and can cause legitimate transactions to fail.

**Vulnerability Details**

In the `_processPayment` function at line 341, when `paymentToken` is `address(0)` (indicating native ETH payment), the contract checks if `msg.value` is *exactly equal* to the calculated price. This strict equality check does not account for scenarios where users might unintentionally send slightly more ETH due to frontend calculation discrepancies, gas price estimates, or simply to ensure the transaction goes through without being short. Minor variations in the value sent, even if over the required price, will cause the transaction to revert with `InvalidPayment()`. This inflexibility leads to a poor user experience.

**Impact**

The overly strict payment validation will result in a frustrating user experience, where users might see their rating attempts fail frequently, even if they intended to pay the correct amount or slightly more. This can discourage platform use, lead to support overhead, and reduce overall adoption, as users may perceive the system as unreliable or difficult to use.

**Recommendation**

Modify the payment validation logic to allow for overpayment and refund any excess amount to the user. This is a common and user-friendly pattern.

```
// In _processPayment function, around line 340
if (paymentToken == address(0)) {
    if (msg.value < price) revert InsufficientPayment(); // Check for underpayment
    if (msg.value > price) {
        uint256 excess = msg.value - price;
        _refundExcessNative(excess); // Assuming _refundExcessNative sends to msg.sender
    }
    _distributePaymentNative(price); // Process the exact price
} else {
    // ... ERC20 logic remains ...
}
```

**Status**

Resolved

# Medium

## [M-01] ImmutableRatings#_executeSwap - Potential Fund Drainage Due to Incorrect Refund Mechanism

### Description

The `_executeSwap` function, when refunding excess native currency after a swap, incorrectly attempts to refund `address(this).balance` instead of the calculated excess from the specific transaction.

If a swap involving native ETH (where `swapParams.token == address(0)`) completes successfully but uses less than `msg.value` (the `amountInMaximum` for ETH swaps), the refund logic at line 414 (`_refundExcessNative(address(this).balance);`) will attempt to send the *entire* ETH balance of the `ImmutableRatings` contract to msg.sender

### Impact

This is a critical vulnerability that could lead to the complete drainage of all ETH held by the contract, including funds from previous transactions meant for the receiver, protocol reserves, or ETH belonging to other users awaiting processing from other function calls (e.g., direct payments, future swaps). This affects the financial integrity of the protocol and can cause significant loss of funds for all stakeholders.

### Recommendation

The refund amount should be precisely calculated as the difference between `msg.value` (the total ETH sent for the swap) and amountIn (the actual ETH consumed by the swap).

```
// In _executeSwap function, around line 412
if (swapParams.token == address(0)) {
    swapRouter.refundETH(); // Router refunds its own excess ETH
    // Calculate specific excess from this transaction
    uint256 excessAmount = msg.value - amountIn;
    if (excessAmount > 0) {
        _refundExcessNative(excessAmount);
    }
```

```
}
```

**Status**

Resolved

### [M-02] ImmutableRatings#initialize&setRatingPrice - Unvalidated Rating Price in Constructor and Setter Function

**Description**

The contract does not validate the `_ratingPrice` in the constructor or in the `setRatingPrice` , allowing it to be set to zero or an otherwise unreasonable value.

Setting the `ratingPrice` to zero either during deployment or through a subsequent call to `setRatingPrice` would allow users to create ratings for free.

**Impact**

it breaks the intended economic model of the platform, potentially leading to spam ratings and devaluing the system. If an operator accidentally sets a very high, unintended price, it could also render the platform unusable until corrected. Lack of validation can lead to economic vulnerabilities or operational disruptions requiring redeployment or urgent intervention by the operator.

**Recommendation**

Implement validation for `_ratingPrice` in both the initialize function and the `setRatingPrice` function to ensure it is not zero and potentially within a reasonable range if applicable.

**Status**

Resolved

# Low

## [L-01] ImmutableRatings# - Violation of Checks-Effects-Interaction Pattern

### Description

The `createDownRatingSwap` function and other such functions perform state changes (`_createDownRating`) before handling external calls for payment processing (`_processPaymentSwap`), violating the CEI pattern.

### Recommendation

Reorder the operations in `createDownRatingSwap` to follow the `Checks-Effects-Interactions` pattern: first, process the payment (interaction), then perform internal state changes (effects).

### Status

Resolved

## [L-02] ImmutableRatings#setReceiever - Missing Zero-Address Validation for Fee Receiver

### Description

The setReceiver function lacks a check to prevent setting the receiver `address` to `address(0)`.

If an operator accidentally or maliciously sets the receiver address to the zero address, all fees collected by the platform through native ETH or ERC20 payments would be transferred to address(0). Funds sent to the zero address are irrecoverable, leading to a permanent loss of all fee revenue generated from that point onward until the receiver is updated to a valid address.

### Recommendation

Add a `zero-address` check in the `setReceiver` function to prevent this scenario.

**Status**

Resolved

### [L-03] ImmutableRatings#setSwapRouter - Swap Router Address Can Be Set Without Validation

**Description**

The `setSwapRouter` function (`ImmutableRatings.sol:193`) allows setting the `swapRouter` address without any validation, such as checking for `address(0)` or if the provided address implements the necessary `IV3SwapRouter` interface.

**Recommendation**

Implement validation in the `setSwapRouter` function. At a minimum, check against `address(0)`. Consider adding a check to ensure the new address supports the `WETH9()` function or other key interface functions

**Status**

Resolved

# QA

## [Q-01] UniversalMappingProtocol - Potentially Unused functions in Contract

### Description

The createMapping function and such other functions in `ImmutableMapping.sol` appear to be unused by the `ImmutableRatings` contract (which uses `safeCreateMappingFor`). This function increases deployment gas costs and adds to the contract's bytecode size unnecessarily.

### Recommendation

Few functions are unused and not intended for future direct external calls; they should be removed to optimize gas costs and reduce contract size. If it has a specific intended purpose not evident from its current usage, this should be documented.

### Status

Acknowledged

## [Q-02] Contracts - Use of Floating Pragma Version

### Description

Both `ImmutableRatings` and `UniversalMappingProtocol` use a floating pragma version (^0.8.22)

### Recommendation

It is best practice to lock the pragma to a specific compiler version that has been thoroughly tested for the project. This ensures consistency across all deployments and development environments.

### Status

Resolved

# Centralisation

The Immutable Ratings project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

# Conclusion

After Hashlock's analysis, the Immutable Ratings project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

# Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

**Manual Code Review:**

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

**Vulnerability Analysis:**

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

**Documenting Results:**

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

**Suggested Solutions:**

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

# Disclaimers

## Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

## Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

# About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

**Website:** hashlock.com.au
**Contact:** info@hashlock.com.au